RotaExpress

Kubernetes na prática: publicar, expor, testar carga e escalar uma aplicação com 1, 2 e 3 Pods

Relatório da atividade

Aluno: Bruno Eduardo Callegaro

Disciplina: DevOps — Operações e Kubernetes

Professor: Lucca Abbado Neres

Instituição: Centro Universitário Fundação Assis Gurgacz – FAG

Data das medições: 24 de outubro de 2026

> **Origem dos dados**
>
> Todos os números das seções 6 e 7 foram **medidos no cluster Minikube** do autor, com o roteiro do Guia da Atividade. Os registros completos do k6 estão em `resultados/` e a tabela em `resultados.csv`.
>
> A seção 5 traz testes feitos **fora do Kubernetes** durante a preparação; eles não entram na comparação. O teste de limite do nó com 6 réplicas (Grupo 8) e o modelo do processador estão marcados em amarelo **[PREENCHER]** até serem registrados.

1. Objetivo

A RotaExpress é uma aplicação que calcula a menor rota de entrega entre um depósito e até nove pontos. Ela funciona bem quando um atendente faz um pedido por vez. A pergunta da atividade é o que acontece quando vários pedidos chegam ao mesmo tempo:

> **Pergunta da investigação**
> Mantendo a mesma demanda, como a quantidade de Pods afeta o número de rotas concluídas, o tempo de resposta e as falhas? Qual é o custo em recursos de cada configuração?

Para responder, a atividade faz o trabalho de quem opera o serviço: preparar o cluster, empacotar a aplicação em uma imagem, escrever os manifestos (Namespace, Deployment, Service e Job), publicar, verificar, provocar uma falha de implantação e voltar atrás, rodar um teste de carga dentro do cluster e comparar 1, 2 e 3 réplicas.

2. Tecnologias utilizadas

Tecnologia	Papel no projeto
Python 3.12 (biblioteca padrão)	Aplicação app.py: servidor HTTP, cálculo por força bruta (9! = 362.880 ordens) e endpoints /healthz e /readyz.
Docker Desktop (WSL 2)	Constrói a imagem rotas:1.0 e serve de base para o cluster local.
Minikube v1.39.0	Cluster Kubernetes de um nó (perfil ops-lab, 3 CPUs, 3 GiB) dentro de um contêiner Docker.
Kubernetes v1.37 / kubectl	Orquestra os Pods. kubectl é o cliente da API usado em todos os passos.
metrics-server	Coleta uso de CPU e memória e alimenta o kubectl top.
Grafana k6 1.0.0	Gerador de carga com taxa de chegada constante, executado como Job dentro do cluster.
ConfigMap	Leva o script carga.js para dentro do Pod do k6, sem construir uma imagem própria.
PowerShell	Scripts ciclo-medicao.ps1 e extrair-resultados.ps1, que executam o ciclo de medição do guia e montam a tabela.

3. Cenário do experimento

Item	Valor
Computador	Windows 10 Pro 22H2; processador AMD [PREENCHER: modelo]; 32 GB de RAM; virtualização (SVM) ativada na BIOS
Cluster	Minikube v1.39.0, Kubernetes v1.37.0, 1 nó, driver Docker, 3 CPUs e 3072 MiB (perfil ops-lab)
Imagem	rotas:1.0 (python:3.12-slim, usuário 10001, sem dependências externas)
Recursos por Pod da aplicação	requests 500m CPU / 64Mi; limits 500m CPU / 128Mi (QoS Burstable)
Recursos do gerador (k6)	requests 250m / 128Mi; limits 500m / 256Mi
Taxa (TAXA)	6 pedidos iniciados por segundo, mantida após o piloto (seção 6.1)
Tamanho da rota (PARADAS)	9 entregas
Duração (DURACAO)	60 s medidos, depois de 10 s de aquecimento e 10 s de intervalo (≈

Item	Valor
	360 tentativas por medição)
Repetições	3 por configuração (rodadas A, B e C), na ordem alternada do passo 7.5 do guia
Variável alterada	Somente o número de réplicas (1, 2 e 3). Imagem, taxa, duração e recursos fixos.

Por que só uma variável muda. Se a taxa, a imagem ou os recursos mudassem entre as execuções, não daria para saber se a diferença veio da quantidade de Pods ou de outra coisa ("mudar uma variável por vez", guia A.2).

4. Procedimentos realizados

A ordem segue os grupos do Guia da Atividade. Cada camada foi testada antes da próxima ("separar domínios de falha").

Grupo	O que foi feito	O que foi observado
0 Ferramentas	Instalação do Docker Desktop (WSL 2), kubectl e Minikube; ativação do SVM Mode na BIOS.	As três ferramentas responderam com versão.
1 Cluster e contexto	minikube start -p ops-lab ..., namespace ops-lab como padrão, metrics-server.	Nó Ready; contexto ops-lab com namespace ops-lab.
2 Imagem	docker build -t rotas:1.0 ., teste com docker run e curl.exe, minikube image load.	Rota calculada; docker.io/library/rotas:1.0 listada no nó.
3 Deployment	Versão mínima, Pod apagado de propósito, depois versão completa (recursos, probes, securityContext).	Pod recriado com outro nome e IP. Rolling update: o Pod antigo (QoS BestEffort) saiu só depois de o novo (QoS Burstable) ficar pronto. 2 revisões.
4 Service	Service ClusterIP rotas:8000, EndpointSlices, wget de dentro do cluster, port-forward.	{"status": "ok", ...} pelo nome rotas; interface calculou rota em http://localhost:8080.
5 Falha e rollback	kubectl set image ... rotas=rotas:1.1 (tag inexistente) e kubectl rollout undo.	Pod novo em ErrImagePull/ImagePullBackOff ("pull access denied, repository does not exist"); o serviço continuou respondendo; o undo voltou a rotas:1.0.
6 Teste de carga	ConfigMap carga-script, Job carga, piloto com 1 Pod.	Piloto válido; TAXA = 6 mantida.
7 Escala e medição	9 medições (3 × 1, 2 e 3 Pods) com scripts/ciclo-medicao.ps1.	Todas com resumo do k6 e 0 iterações descartadas.
8 Diagnóstico e decisão	Análise (seções 6–9). Teste com 6 réplicas e encerramento.	[PREENCHER: resultado do teste com 6 réplicas]

4.1 Manifestos escritos

k8s/namespace.yaml cria o namespace `ops-lab`, onde vivem todos os objetos da atividade.

k8s/deployment.yaml (versão completa do Apêndice B) declara 1 réplica do contêiner `rotas` com a imagem `rotas:1.0` e `imagePullPolicy: IfNotPresent`, selector `app: rotas` e:

- **Recursos:** requests e limits de 500m de CPU, para que cada cópia tenha exatamente meia CPU; memória de 64Mi (request) a 128Mi (limit).
- **readinessProbe** em `/readyz` a cada 5 s: o Pod só recebe tráfego quando responde.
- **livenessProbe** em `/healthz` a cada 10 s, com 10 s de espera inicial: reinicia o contêiner se o processo travar.

- **securityContext:** não roda como root, sem escalada de privilégio, sistema de arquivos somente leitura.

A primeira versão está em `referencia/deployment-v1.yaml`; aplicar uma e depois a outra gerou as duas revisões do passo 3.6.

k8s/service.yaml cria o Service `rotas` (ClusterIP, porta 8000 → 8000, selector `app: rotas`), o nome estável usado pelo k6.

k8s/job-carga.yaml cria o Job `carga` (`grafana/k6:1.0.0`, `backoffLimit: 0`, `activeDeadlineSeconds: 240`, `restartPolicy: Never`), com TAXA, PARADAS e DURACAO como variáveis, recursos próprios e o ConfigMap `carga-script` em `/scripts`. A label do Pod é `app: gerador-carga`, para o Service não enviar tráfego ao gerador.

Os manifestos foram validados com o kubeconform em modo estrito (5 recursos, 5 válidos) e aplicados sem erro no cluster.

5. Validação feita na preparação (fora do Kubernetes)

> **Não faz parte da comparação**
> VM Linux de 2 vCPUs, sem Kubernetes, um processo do `app.py` limitado a 500m por cgroup. Arquivos em `validacao_local/`.

Cálculo com 9 paradas: 218 a 326 ms sem limite de CPU e 418 a 586 ms com 500m. Dois pedidos simultâneos geraram um HTTP 200 e um HTTP 503. O `carga.js` contra um processo, com TAXA = 6, teve 1,75 sucesso/s e 70,91% de falhas. Esses números anteciparam a ordem de grandeza confirmada no cluster (seção 6).

6. Resultados das medições no Kubernetes

6.1 Piloto e calibração

Métrica (piloto, 1 Pod, TAXA = 6)	Valor
rotas_ok	120 (2,00 sucessos/s)
falhas	66,66% (240 de 360 tentativas)
latencia_ok	média 399,39 ms; p95 431,51 ms; máx. 531,7 ms
dropped_iterations{scenario:medicao}	0
Job	Failed, k6 com código 99 (meta de falhas não atingida)

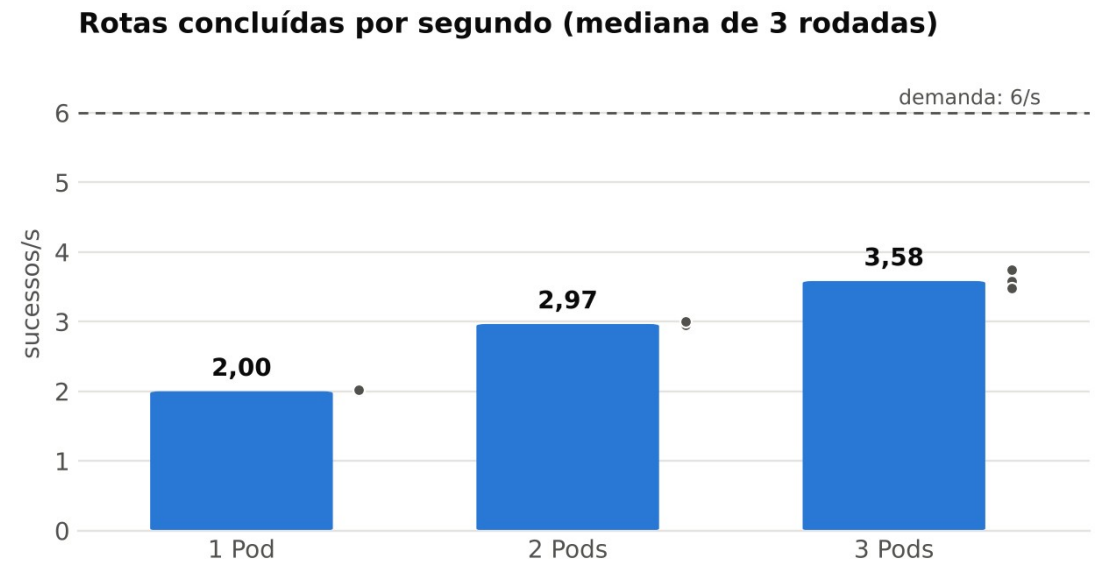
Pelo critério do passo 6.6, houve falhas, mas também uma quantidade razoável de sucessos e nenhuma iteração descartada. A taxa de **6 pedidos/s** foi mantida para toda a série. Ela fica entre a capacidade estimada de 1 Pod (≈ 2/s) e a de 3 Pods (≈ 6/s), faixa que o guia indica como informativa.

6.2 Tabela comparativa (medianas das rodadas A, B e C)

Métrica	1 Pod	2 Pods	3 Pods
Rotas concluídas (rotas_ok)	120	178	215
Sucessos por segundo (rotas_ok / 60)	2,00	2,97	3,58
Falhas (% das tentativas, inclui 503)	66,75%	50,69%	40,44%
Latência média das respostas OK	428,8 ms	425,6 ms	430,3 ms
Latência p95 das respostas OK	445,4 ms	446,8 ms	449,6 ms
dropped_iterations (medição)	0	0	0
Status do Job	Failed (99)	Failed (99)	Failed (99)

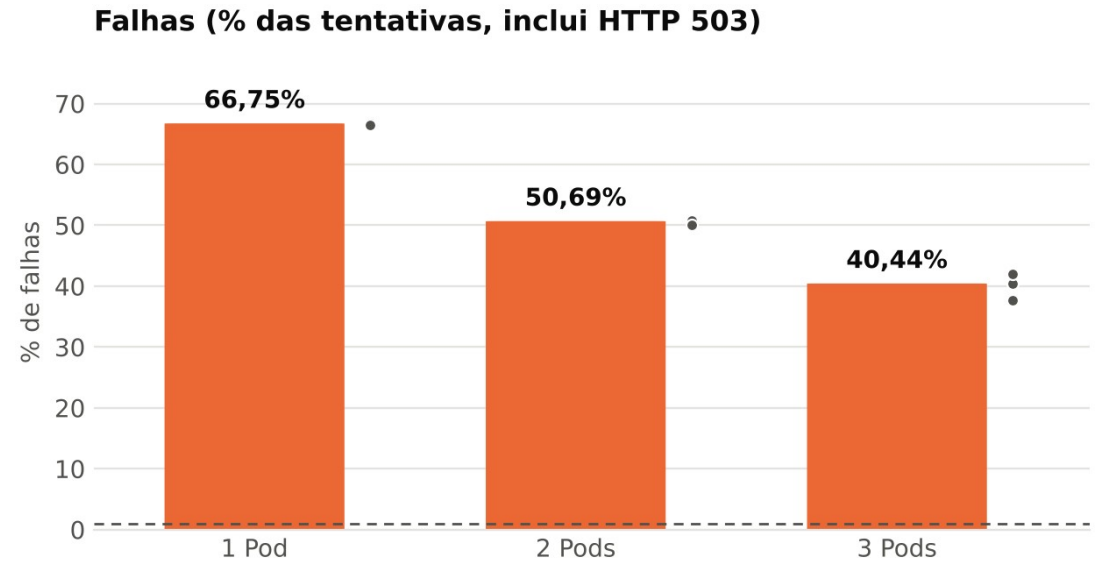
Métrica	1 Pod	2 Pods	3 Pods
CPU reservada (requests)	500m	1000m	1500m
Sucessos/s por CPU reservada	4,00	2,97	2,39
Ganho de sucessos/s vs 1 Pod	1,00×	1,48×	1,79×
Atende às 3 metas?	Não	Não	Não

Metas: falhas < 1%, p95 de latencia_ok < 800 ms e nenhuma iteração descartada. A latência e as iterações descartadas ficaram dentro da meta em todas as medições; a meta de falhas não foi atingida em nenhuma.



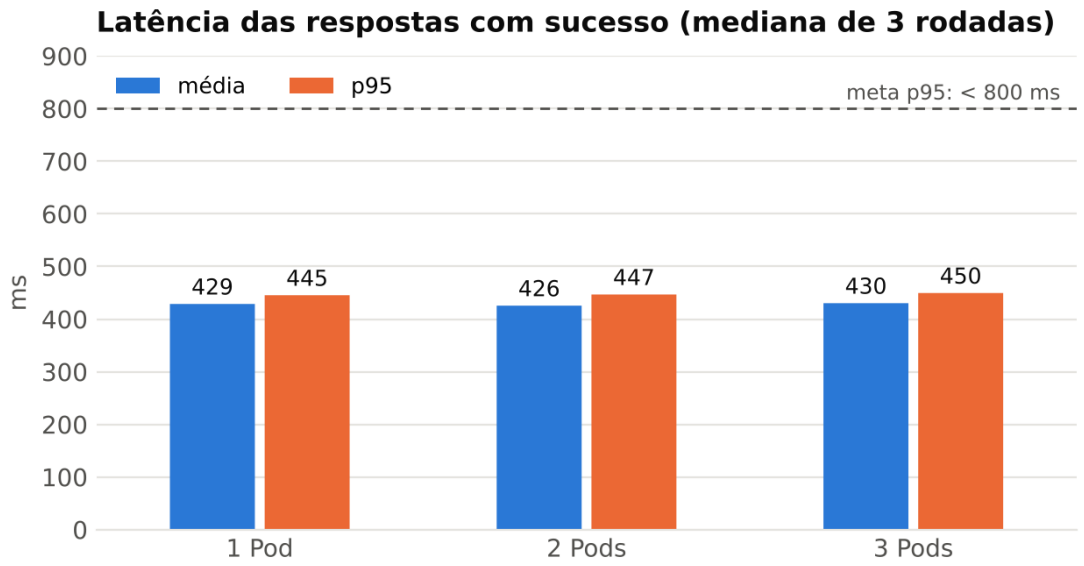
Pontos cinza: cada rodada (A, B, C). Fonte: resultados.csv, TAXA = 6, 9 paradas, 60 s medidos.

Figura 1 — Sucessos por segundo. Barras: mediana; pontos: cada rodada.



Pontos cinza: cada rodada (A, B, C). Linha tracejada: meta de 1%, não atingida por nenhuma configuração.

Figura 2 — Percentual de falhas (respostas sem rota válida, incluindo 503).



Só respostas HTTP 200 com rota válida (latencia_ok). As recusas 503 não entram neste tempo.

Figura 3 — Latência média e p95 das respostas com sucesso.

6.3 Todas as medições

Medição	Pods	rotas_ok	Sucessos/s	Falhas	p95 (ms)	Média (ms)	Dropped
3pods-a	3	215	3,58	40,44%	472,9	421,3	0
1pod-a	1	120	2,00	66,75%	434,4	428,8	0
2pods-a	2	178	2,97	50,69%	482,5	425,6	0
2pods-b	2	177	2,95	50,83%	446,1	425,6	0
3pods-b	3	225	3,75	37,67%	449,6	430,3	0
1pod-b	1	120	2,00	66,75%	481,0	420,2	0
1pod-c	1	121	2,02	66,48%	445,4	433,6	0
2pods-c	2	180	3,00	50,13%	446,8	426,8	0
3pods-c	3	209	3,48	41,94%	446,1	432,3	0

As repetições ficaram muito próximas entre si: 1 Pod variou de 2,00 a 2,02 sucessos/s, 2 Pods de 2,95 a 3,00 e 3 Pods de 3,48 a 3,75. A ordem alternada das rodadas não mudou o resultado, o que dá confiança na mediana.

6.4 Distribuição entre os Pods e uso de recursos

Medição	Aceitos / recusados por Pod (depois − antes)	CPU máx. observada (kubectl top)
1pod-a	+140 / −280	355m
2pods-a	+102/−110 · +106/−103	185m · 228m
2pods-c	+107/−106 · +104/−105	469m · 233m
3pods-a	+78/−47 · +81/−56 · +94/−66	194m · 171m · 198m
3pods-b	+91/−49 · +84/−59 · +85/−53	227m · 216m · 240m
3pods-c	+81/−60 · +81/−60 · +79/−59	158m · 295m · 247m

O Service dividiu a carga de forma equilibrada: nas medições com 3 Pods, cada réplica aceitou entre 78 e 94 pedidos. Os contadores incluem o aquecimento, por isso somam um pouco mais que rotas_ok. A memória ficou entre 13 e 15 MiB por Pod no piloto, muito abaixo do request de 64Mi.

O `kubectl top` mostrou valores abaixo de 500m porque o metrics-server publica uma média de cerca de um minuto, e essa janela incluía parte do tempo sem carga. As leituras servem como indício (guia, 7.3), não como medida de saturação.

6.5 Rollback e limite do nó

Rollback (Grupo 5): com `rotas:1.1`, o Pod novo ficou em `ErrImagePull` e depois `ImagePullBackOff`, com o evento "pull access denied, repository does not exist". Como ele nunca ficou pronto, não entrou no Service; o `wget` para `http://rotas:8000/healthz` continuou respondendo `ok` pelo Pod antigo. `kubectl rollout undo` restaurou `rotas:1.0`.

Limite do nó (Grupo 8): [PREENCHER: com 6 réplicas, quantos Pods ficaram `Pending` e o evento `FailedScheduling ... Insufficient cpu`].

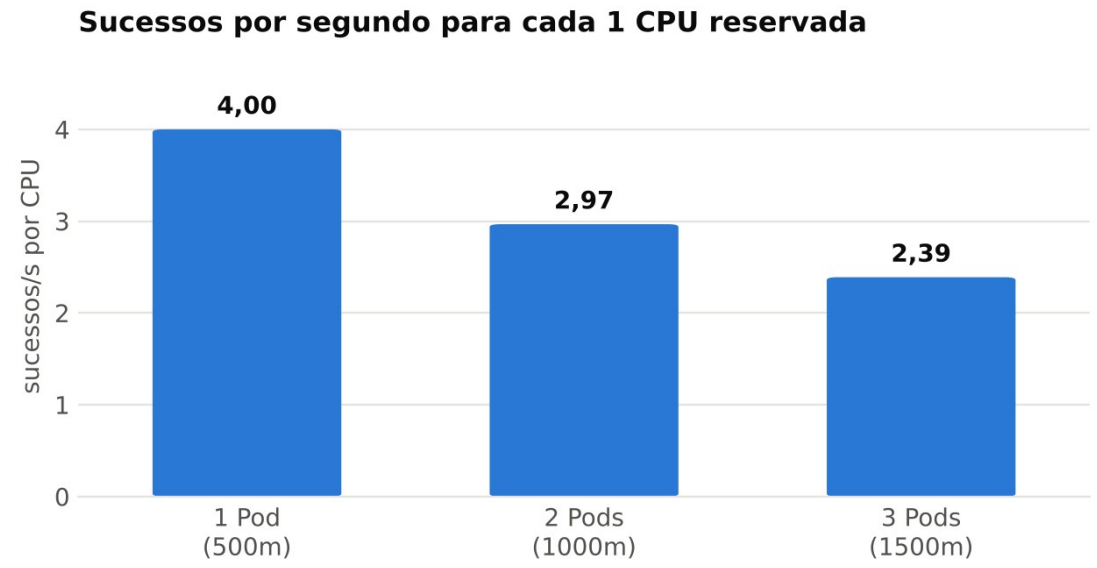
7. Análise

7.1 Por que 1 Pod atende exatamente 2 pedidos por segundo

O k6 inicia um pedido a cada 1/6 s (≈ 167 ms). Cada cálculo leva cerca de 0,42 a 0,43 s com meia CPU (média de 429 ms nos sucessos). Quando um Pod aceita um pedido, os dois seguintes chegam com ele ainda ocupado e recebem 503; o terceiro, 0,5 s depois, encontra o Pod livre. Por isso 1 Pod aceitou **1 de cada 3 pedidos**: 2,00 sucessos/s e 66,7% de falhas. Os contadores confirmam: cerca de +140 aceitos e −280 recusados em cada medição de 1 Pod.

7.2 Mais réplicas ajudaram, mas menos que o proporcional

- **Sucessos subiram:** 2,00 → 2,97 → 3,58 por segundo (+48% com 2 Pods, +79% com 3). Havia trabalho que podia ser feito em paralelo, porque cada cálculo é independente e só usa CPU.
- **Mas não triplicaram.** Pela capacidade, 3 Pods deveriam atender perto de 6 pedidos/s. A causa está na forma de distribuir: o Service escolhe o Pod de cada conexão sem saber se ele está ocupado. Mesmo com uma réplica livre, um pedido pode cair em outra que está calculando e receber 503 (guia, 8.3). Quanto mais réplicas, mais pedidos "batem" em Pods ocupados com outros livres ao lado.
- **O custo por resultado subiu:** cada CPU reservada rendeu 4,00 sucessos/s com 1 Pod, 2,97 com 2 e 2,39 com 3 (Figura 4). O segundo Pod acrescentou cerca de 0,97 sucesso/s; o terceiro, só 0,62.



Mediana de sucessos/s dividida pela CPU reservada (réplicas × 500m de requests).

Figura 4 — Sucessos por segundo para cada 1 CPU reservada.

7.3 A sobrecarga apareceu como falha, não como lentidão

A latência dos sucessos ficou praticamente igual nas três configurações (média 426–430 ms, p95 445–450 ms), bem abaixo da meta de 800 ms. Sem fila, quem é atendido é atendido logo, e quem chega com o Pod ocupado recebe 503 na hora. Por isso o p95 sozinho daria uma impressão errada: ele mede só quem conseguiu rota. É o caso "p95 baixo, mas muitas falhas" da tabela do guia (8.3).

A latência estável também indica que **não houve disputa de CPU no nó** com 3 Pods: se o nó estivesse sobrecarregado, cada cálculo ficaria mais lento com mais réplicas, e isso não aconteceu. A queda de eficiência vem da distribuição, não da falta de CPU.

7.4 O que foi observado e o que é hipótese

Observado nos dados	Hipótese (não testada aqui)
Sucessos crescem com as réplicas, de forma menos que proporcional.	Um balanceamento que conhece a ocupação (ou uma pequena fila em cada Pod) faria 3 Pods se aproximarem de 6 pedidos/s.
Falhas caem de 67% para 40%, mas continuam muito acima de 1%.	Só adicionar réplicas, com a aplicação como está, não leva as falhas abaixo de 1% em um número razoável de Pods.
Latência dos sucessos estável em ≈ 430 ms.	Com fila, as falhas cairiam, mas a latência subiria; seria preciso medir de novo contra a meta de 800 ms.

8. Dificuldades e limitações

- **Preparação do computador:** foi preciso instalar o WSL (`wsl --install`) e ativar o SVM Mode na BIOS (placa Gigabyte AORUS) para o Docker Desktop funcionar.
- **Um único nó.** Três Pods no mesmo nó não protegem contra a queda desse nó. O laboratório mede atendimento sob carga, não recuperação de desastre (guia, 8.4).
- **Balanceamento por conexão, sem saber a ocupação.** É consequência do desenho da aplicação (um cálculo por Pod, sem fila), não uma limitação do Kubernetes.
- **kubectl top atrasado.** As leituras são médias de cerca de um minuto e subestimam o pico; também registraram Pods em encerramento logo após reduzir réplicas, que foram desconsiderados na tabela.
- **O gerador divide o nó com a aplicação.** O k6 tem limite de 500m; a latência estável sugere que isso não distorceu os resultados.
- **A taxa calibrada vale para este computador.** Outras máquinas podem exigir outra TAXA.

9. Conclusão

Nenhuma configuração atingiu as três metas. A latência (p95 ≈ 450 ms) e as iterações descartadas (0) ficaram dentro da meta em todas, mas as falhas ficaram em 66,75% com 1 Pod, 50,69% com 2 e 40,44% com 3, muito acima de 1%.

Entre as testadas, a melhor foi **3 Pods**: 3,58 sucessos/s, 1,79 vez o resultado de 1 Pod, com 1500m de CPU reservada. Para a demanda de 6 pedidos/s, **adotaria 3 Pods**, sabendo que ela ainda recusa cerca de 4 em cada 10 pedidos e que cada CPU a mais rende menos que a anterior.

A principal lição é que **mais réplicas ajudam quando o trabalho pode ser feito em paralelo, mas não resolvem tudo**: o desenho da aplicação (um cálculo por vez, sem fila) e a forma como o Service distribui as conexões limitaram o ganho. A próxima investigação seria testar uma fila pequena na frente do cálculo ou um balanceamento que considere a ocupação, medir de novo contra as mesmas metas e, só depois, configurar um HorizontalPodAutoscaler com base nessa capacidade conhecida.

10. Encerramento

```
kubectl scale deployment/rotas --replicas=6   # Grupo 8: limite do nó
```

```
kubectl get pods
kubectl describe node ops-lab
kubectl scale deployment/rotas --replicas=1
kubectl delete namespace ops-lab
minikube stop
```

`kubectl delete namespace ops-lab` apaga, em cascata, Deployment, ReplicaSets, Pods, Service, ConfigMap e Jobs. Os manifestos continuam em `k8s/` para recriar tudo.

Referências

- NERES, Lucca Abbado. RotaExpress — Kubernetes na prática (Guia da Atividade). FAG, 2026.
- NERES, Lucca Abbado. Operações, contêineres e Kubernetes (material de aula). FAG, 2026.
- Kubernetes. Deployments; Service; Jobs; kubectl Quick Reference. https://kubernetes.io/docs/ (consulta: 15 out. 2026).
- Minikube. Get Started. https://minikube.sigs.k8s.io/docs/start/ (consulta: 15 out. 2026).
- Grafana k6. Constant arrival rate executor. https://grafana.com/docs/k6/latest/ (consulta: 15 out. 2026).